

Seletores Avançados, Pseudo-Elementos e Animações

Aprofundando em CSS

Revisão: Seletores Básicos

- **Seletor Universal** (`*`): Selecciona todos os elementos.
- **Seletor de Tipo** (`h1` , `p`): Selecciona elementos pelo nome da tag.
- **Seletor de Classe** (`.classe`): Selecciona elementos com a classe especificada.
- **Seletor de ID** (`#id`): Selecciona o elemento com o ID especificado.

```
* { margin: 0; padding: 0; }  
p { color: blue; }  
.destaque { font-weight: bold; }  
#cabecalho { background: #333; }
```

Seletores de Descendência e Filhos

Seletor de Descendência ()

Seleciona todos os elementos dentro de um ancestral.

```
div p { color: red; } /* Todos os <p> dentro de <div> */
```

Seletor de Filho Direto (>)

Seleciona apenas filhos diretos.

```
div > p { color: blue; } /* Apenas <p> filhos diretos de <div> */
```

Seletores de Irmãos

Irmão Adjacente (+)

Seleciona o irmão imediatamente seguinte.

```
h2 + p { font-size: 1.2em; } /* Primeiro <p> após <h2> */
```

Irmãos Gerais (~)

Seleciona todos os irmãos seguintes.

```
h2 ~ p { color: gray; } /* Todos os <p> após <h2> */
```

Seletores de Atributo

Permitem selecionar elementos com base em seus atributos HTML.

```
/* Elementos com o atributo target */  
a[target] { color: red; }  
  
/* Atributo com valor exato */  
a[target="_blank"] { color: blue; }  
  
/* Atributo que começa com o valor */  
a[href^="https"] { color: green; }  
  
/* Atributo que termina com o valor */  
a[href$=".pdf"] { color: orange; }  
  
/* Atributo que contém o valor */  
a[href*="github"] { color: purple; }
```

Pseudo-Classes de Estado

Controlam o estilo com base no estado do elemento.

```
/* Quando o mouse está sobre o elemento */  
a:hover { color: red; text-decoration: underline; }  
  
/* Quando o elemento está focado */  
input:focus { border-color: blue; outline: none; }  
  
/* Quando o link já foi visitado */  
a:visited { color: purple; }  
  
/* Quando o elemento está ativo (clicado) */  
a:active { color: orange; }  
  
/* Elemento desabilitado */  
input:disabled { opacity: 0.5; cursor: not-allowed; }
```

Pseudo-Classes Estruturais

Selecionam elementos com base na posição na árvore DOM.

```
/* Primeiro filho */
li:first-child { font-weight: bold; }

/* Último filho */
li:last-child { border-bottom: none; }

/* Enésimo filho */
li:nth-child(odd) { background: #f0f0f0; }
li:nth-child(even) { background: #fff; }
li:nth-child(3n) { color: red; } /* A cada 3 elementos */

/* Primeiro do tipo */
p:first-of-type { font-size: 1.5em; }

/* Último do tipo */
p:last-of-type { margin-bottom: 0; }
```

Pseudo-Classes de Formulário

Específicas para elementos de formulário.

```
/* Campo obrigatório */  
input:required { border-left: 3px solid red; }  
  
/* Campo válido */  
input:valid { border-color: green; }  
  
/* Campo inválido */  
input:invalid { border-color: red; }  
  
/* Checkbox marcado */  
input:checked + label { font-weight: bold; }  
  
/* Placeholder visível */  
input:placeholder-shown { color: #999; }
```


Pseudo-Elementos

Criam elementos virtuais no DOM para estilização.

::before e **::after**

Inserem conteúdo antes/depois do conteúdo real.

```
/* Adiciona aspas antes e depois do texto */
blockquote::before { content: open-quote; font-size: 2em; }
blockquote::after { content: close-quote; font-size: 2em; }

/* Adiciona um ícone antes de links externos */
a[href^="http"]::before {
  content: "🔗 ";
}
```

Pseudo-Elementos: `::first-letter` e `::first-line`

```
/* Estiliza a primeira letra */  
p::first-letter {  
  font-size: 3em;  
  font-weight: bold;  
  float: left;  
  margin-right: 8px;  
  line-height: 1;  
}  
  
/* Estiliza a primeira linha */  
p::first-line {  
  font-variant: small-caps;  
  font-weight: bold;  
}
```

Pseudo-Elemento `::selection`

Controla a aparência do texto selecionado.

```
::selection {  
  background-color: #3498db;  
  color: white;  
}  
  
/* Seleção específica para elementos */  
p::selection {  
  background-color: #e74c3c;  
  color: white;  
}
```

Pseudo-Elemento `::placeholder`

Estiliza o texto placeholder de inputs.

```
input::placeholder {  
  color: #aaa;  
  font-style: italic;  
  font-size: 0.9em;  
}  
  
textarea::placeholder {  
  color: #bbb;  
}
```

Pseudo-Elemento `::marker`

Estiliza o marcador de listas.

```
li::marker {  
  color: #e74c3c;  
  font-size: 1.5em;  
}  
  
/* Marcadores diferentes para listas ordenadas */  
ol li::marker {  
  color: #3498db;  
  font-weight: bold;  
}
```

Combinando Seletores e Pseudo-Elementos

```
/* Card com ícone e borda animada */
.card {
  position: relative;
  padding: 20px;
  border: 2px solid #ddd;
  transition: border-color 0.3s;
}

.card:hover {
  border-color: #3498db;
}

.card::before {
  content: "";
  position: absolute;
  top: -2px; left: -2px;
  right: -2px; bottom: -2px;
  border: 2px solid #3498db;
  opacity: 0;
  transition: opacity 0.3s;
}

.card:hover::before {
  opacity: 1;
}
```

Introdução às Animações CSS

Transições (**transition**)

Animam mudanças de propriedades de forma suave.

```
.botao {  
  background-color: #3498db;  
  color: white;  
  padding: 10px 20px;  
  border: none;  
  border-radius: 5px;  
  transition: all 0.3s ease;  
}  
  
.botao:hover {  
  background-color: #2980b9;  
  transform: scale(1.05);  
  box-shadow: 0 4px 8px rgba(0,0,0,0.2);  
}
```

Propriedades de Transição

```
.elemento {  
  /* transition: propriedade duração timing-function delay; */  
  transition: background-color 0.5s ease-in-out 0.2s;  
  
  /* Múltiplas transições */  
  transition:  
    background-color 0.3s ease,  
    transform 0.5s cubic-bezier(0.175, 0.885, 0.32, 1.275),  
    box-shadow 0.3s ease;  
}
```


Funções de Timing

- `ease` (padrão): Início lento, acelera, desacelera no final.
- `linear` : Velocidade constante.
- `ease-in` : Início lento.
- `ease-out` : Final lento.
- `ease-in-out` : Início e final lentos.
- `cubic-bezier()` : Controle personalizado.

Animações com `@keyframes`

Permitem criar animações complexas e personalizadas.

```
/* Definição da animação */
@keyframes pulsar {
  0% { transform: scale(1); }
  50% { transform: scale(1.1); }
  100% { transform: scale(1); }
}

/* Aplicação da animação */
.coracao {
  animation: pulsar 1s ease-in-out infinite;
}
```

Propriedades de Animação

```
.elemento {  
  /* animation: nome duração timing-function delay iteração direção fill-mode play-state; */  
  animation: pulsar 2s ease-in-out 0.5s infinite alternate forwards;  
  
  /* Múltiplas animações */  
  animation:  
    pulsar 1s ease-in-out infinite,  
    girar 2s linear infinite;  
}
```

Propriedades Individuais

- `animation-name` : Nome da `@keyframes` .
- `animation-duration` : Duração da animação.
- `animation-timing-function` : Função de tempo.
- `animation-delay` : Atraso antes de iniciar.
- `animation-iteration-count` : Número de repetições (`infinite`).
- `animation-direction` : Direção (`normal` , `reverse` , `alternate` , `alternate-reverse`).
- `animation-fill-mode` : Estado antes/depois (`forwards` , `backwards` , `both`).
- `animation-play-state` : Controle (`running` , `paused`).

Exemplo: Animação de Carregamento

```
/* Spinner de carregamento */  
@keyframes girar {  
  0% { transform: rotate(0deg); }  
  100% { transform: rotate(360deg); }  
}  
  
.spinner {  
  width: 40px;  
  height: 40px;  
  border: 4px solid #f3f3f3;  
  border-top: 4px solid #3498db;  
  border-radius: 50%;  
  animation: girar 1s linear infinite;  
}
```

Exemplo: Animação de Fade In

```
@keyframes fadeIn {  
  from {  
    opacity: 0;  
    transform: translateY(20px);  
  }  
  to {  
    opacity: 1;  
    transform: translateY(0);  
  }  
}  
  
.card {  
  animation: fadeIn 0.6s ease-out forwards;  
}  
  
/* Atraso escalonado para múltiplos cards */  
.card:nth-child(1) { animation-delay: 0.1s; }  
.card:nth-child(2) { animation-delay: 0.2s; }  
.card:nth-child(3) { animation-delay: 0.3s; }
```

Exemplo: Animação de Shake (Vibração)

```
@keyframes shake {  
  0%, 100% { transform: translateX(0); }  
  10%, 30%, 50%, 70%, 90% { transform: translateX(-5px); }  
  20%, 40%, 60%, 80% { transform: translateX(5px); }  
}  
  
.erro {  
  animation: shake 0.5s ease-in-out;  
  border-color: red;  
}
```

Exemplo: Animação de Borda Gradiente

```
@keyframes borda-gradiente {  
  0% { border-color: #3498db; }  
  33% { border-color: #e74c3c; }  
  66% { border-color: #2ecc71; }  
  100% { border-color: #3498db; }  
}  
  
.card-animado {  
  border: 3px solid #3498db;  
  animation: borda-gradiente 3s linear infinite;  
}
```


Transformações 2D

```
/* Transladar (mover) */
.elemento { transform: translate(50px, 100px); }

/* Rotacionar */
.elemento { transform: rotate(45deg); }

/* Escalar */
.elemento { transform: scale(1.5); }

/* Inclinar */
.elemento { transform: skew(10deg, 20deg); }

/* Combinando transformações */
.elemento {
  transform: translate(50px, 0) rotate(45deg) scale(1.2);
}
```

Transformações 3D

```
/* Perspectiva no container */  
.container {  
  perspective: 1000px;  
}  
  
/* Rotação 3D */  
.card-3d {  
  transform: rotateY(180deg);  
  transform-style: preserve-3d;  
  transition: transform 0.6s;  
}  
  
.card-3d:hover {  
  transform: rotateY(0deg);  
}
```

Boas Práticas para Animações

✓ Use `transform` e `opacity`

- São mais performáticas (GPU-accelerated).
- Não causam reflow/repaint.

✗ Evite animar

- `width`, `height`, `top`, `left`, `margin`, `padding`.
- Causam reflow e impactam a performance.

🎯 Dicas

- Use `will-change` para elementos que serão animados.
- Prefira `transform` em vez de `top` / `left`.

Acessibilidade: `prefers-reduced-motion`

Respeita a preferência do usuário por menos animações.

```
@media (prefers-reduced-motion: reduce) {  
  *,  
  *::before,  
  *::after {  
    animation-duration: 0.01ms !important;  
    animation-iteration-count: 1 !important;  
    transition-duration: 0.01ms !important;  
  }  
}
```

Exemplo Prático: Botão Animado

```
.botao-animado {  
  position: relative;  
  padding: 12px 24px;  
  background: #3498db;  
  color: white;  
  border: none;  
  border-radius: 5px;  
  overflow: hidden;  
  transition: transform 0.3s, box-shadow 0.3s;  
}
```

```
.botao-animado::before {  
  content: "";  
  position: absolute;  
  top: 0; left: -100%;  
  width: 100%; height: 100%;  
  background: linear-gradient(  
    90deg,  
    transparent,  
    rgba(255,255,255,0.3),  
    transparent  
  );  
  transition: left 0.5s;  
}
```

```
.botao-animado: hover::before {  
  left: 100%;  
}  
  
.botao-animado: hover {  
  transform: translateY(-2px);  
  box-shadow: 0 4px 12px rgba(52,152,219,0.4);  
}
```

Exemplo Prático: Card com Efeito Hover

```
.card-hover {  
  position: relative;  
  padding: 20px;  
  background: white;  
  border-radius: 10px;  
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);  
  transition: transform 0.3s, box-shadow 0.3s;  
  overflow: hidden;  
}  
  
.card-hover::after {  
  content: "";  
  position: absolute;  
  bottom: 0; left: 0;  
  width: 100%; height: 3px;  
  background: linear-gradient(90deg, #3498db, #e74c3c);  
  transform: scaleX(0);  
  transform-origin: left;  
  transition: transform 0.3s;  
}
```

```
.card-hover:hover {  
  transform: translateY(-5px);  
  box-shadow: 0 8px 25px rgba(0,0,0,0.15);  
}  
  
.card-hover:hover::after {  
  transform: scaleX(1);  
}
```


Bora praticar!

Desafio: Crie um componente interativo

1. Card de perfil com:

- Foto com borda animada
- Nome com efeito de sublinhado animado
- Botão com efeito de brilho no hover
- Animação de entrada (fade in + slide up)

2. Menu de navegação com:

- Links com pseudo-elemento `::after` animado
- Efeito de hover suave
- Indicador de página ativa

Recursos Extras

- [MDN: Seletores CSS](#)
- [MDN: Pseudo-elementos](#)
- [MDN: Animações CSS](#)
- [CSS-Tricks: Guia de Seletores](#)
- [Animate.css](#)

Dúvidas?

Allan da Silva